

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

|                |                                                                                                                    |               |                                                       |
|----------------|--------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------------|
| Course Code:   | CSA10                                                                                                              | Course Title: | Software Engineering                                  |
| CO Assessed:   | CO3                                                                                                                | Assessment:   | Assessment Tool 1 – Git & Docker Technical Assignment |
| Weightage:     | 20%                                                                                                                | Total Marks:  | 25                                                    |
| CO3 Statement: | Build full-stack applications with an emphasis on containerization, version control, and collaborative development |               |                                                       |
| Student Name:  | N.RAMYA                                                                                                            | Reg. No.:     | 192511403                                             |
| Date:          | 13.08.2026                                                                                                         | Duration:     | 60 minutes                                            |

### Code of Conduct

I N.RAMYA(192511403)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.RAMYA

## Intelligent AI-Based Smart Public Transportation Demand Forecasting System

### Annexure A – Git & Docker Technical Assignment

### PROBLEM STATEMENT

Public transportation often faces **overcrowding during peak hours and underutilization during off-peak hours** due to inaccurate demand prediction. An AI-based system is needed to **forecast passenger demand and optimize transport schedules and fleet allocation** using historical and real-time data.

### Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

### Assignment Questions

#### 1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

#### 2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. **Git Repository Initialization**
  - Create a Git repository for the project.
  - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
  - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
  - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
  - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
  - Explain the purpose of each Git operation performed.
6. **Commit History Analysis**
  - Present the commit history of the project.
  - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
  - Explain why Docker is suitable for the assigned project.
  - Identify the application components that require containerization.
8. **Dockerfile Development**
  - Design and explain the Dockerfile required to containerize the application.
  - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
  - Develop a Docker Compose configuration for the project.
  - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
  - Demonstrate the execution of the application using Docker containers.
  - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
  - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
  - Identify the challenges encountered during implementation.
  - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

#### Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

#### Rubrics for Calculation

| Evaluation Criteria                                             | Excellent (4 Marks)                                                                                                                                       | Good (3 Marks)                                                       | Satisfactory (2 Marks)                                              | Needs Improvement (1 Mark)                                     |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Problem Analysis &amp; Project Design(20%)</b>               | Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.      | Good analysis and project structure with minor omissions.            | Basic analysis and repository structure with limited justification. | Incomplete or unclear analysis and project organization.       |
| <b>Git Repository &amp; Branching Strategy(20%)</b>             | Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.                                 | Repository and branching strategy are mostly correct.                | Basic Git setup with limited branching implementation.              | Incorrect or incomplete Git repository and branching strategy. |
| <b>Version Control Workflow &amp; Commit History(10%)</b>       | Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.           | Most Git operations demonstrated correctly with good commit history. | Limited Git workflow and commit practices.                          | Poor workflow and inadequate commit history.                   |
| <b>Docker Implementation(20%)</b>                               | Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies. | Functional Docker implementation with minor issues.                  | Basic Docker implementation with limited explanation.               | Incomplete or incorrect Docker implementation.                 |
| <b>Deployment, Testing &amp; Results(15%)</b>                   | Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.                                           | Deployment completed with minor issues.                              | Partial deployment and testing demonstrated.                        | Deployment unsuccessful or insufficient evidence.              |
| <b>Documentation, Challenges &amp; Future Enhancements(15%)</b> | Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.                    | Good documentation with minor omissions.                             | Basic documentation with limited discussion.                        | Poor documentation and insufficient analysis.                  |

| Criteria                                                     | Mark       |
|--------------------------------------------------------------|------------|
| System Requirement Analysis (30%)                            | /5         |
| Selection of Software Architecture (25%)                     | /5         |
| Architecture Diagram and Component Design (20%)              | /5         |
| Component Interaction and Quality Attribute Analysis (15%)   | /5         |
| Architectural Justification and Technical Presentation (10%) | /5         |
| <b>TOTAL</b>                                                 | <b>/25</b> |



**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

**Department of Computer Science and Engineering**

**CO3 ASSESSMENT TOOL 1 – GIT & DOCKER TECHNICAL ASSIGNMENT**

***Technical Report***

**Intelligent AI-Based Smart Public Transportation**

**Demand Forecasting System**

| Course Code                    | CSA10 — Software Engineering                                                                                             |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| CO Assessed                    | CO3 — Build full-stack applications with an emphasis on containerization, version control, and collaborative development |
| Assessment                     | Assessment Tool 1 – Git & Docker Technical Assignment                                                                    |
| Weightage                      | 20%   Total Marks: 25                                                                                                    |
| Assigned Problem Statement No. | 20 — Intelligent AI-Based Smart Public Transportation Demand Forecasting System                                          |
| Student Name                   | N.Ramya                                                                                                                  |
| Register Number                | 192511403                                                                                                                |
| Date                           | 13/08/2026                                                                                                               |

# 1. Problem Analysis

## 1.1 Assigned Problem Statement

Public transportation agencies are responsible for delivering reliable, efficient, and comfortable travel services to commuters while continuously managing fluctuations in passenger demand. In most cities, buses and trains are overcrowded during peak commuting hours (morning and evening office hours) while running largely empty during off-peak periods. This mismatch occurs because traditional scheduling is based on fixed timetables built from historical averages rather than on real-time or predictive analysis of demand. Traditional approaches therefore lack the predictive intelligence and real-time adaptability required to respond to fluctuating ridership caused by factors such as weather, local events, holidays, and traffic congestion.

The assigned project, the Intelligent AI-Based Smart Public Transportation Demand Forecasting System, addresses this gap by building an AI-powered platform that analyses historical ridership data, weather conditions, local events, and traffic patterns to forecast passenger demand accurately. Based on these forecasts, the system recommends optimised bus/train schedules, improves fleet allocation across routes, and provides real-time passenger information so that commuters and operators can both make informed decisions.

## 1.2 Project Objectives

Forecast passenger demand accurately using AI/ML models trained on historical and contextual data.

Optimize public transport schedules dynamically based on predicted demand.

Improve fleet allocation by redistributing buses/trains to high-demand routes.

Monitor passenger flow in real time using GPS, IoT sensors, and ticketing data.

Analyze travel patterns to understand commuter behaviour across time, routes, and seasons.

Generate transportation analytics and dashboards for operators and city planners.

Reduce overcrowding during peak hours through predictive resource redistribution.

Improve overall public transport efficiency and reduce operational wastage.

## 1.3 Functional Requirements

- The system shall ingest historical ridership records, GPS/AVL vehicle-tracking data, weather data, event calendars, and traffic data.
- The system shall pre-process and store the ingested data in a structured, queryable format.
- The system shall train and serve a machine-learning demand-forecasting model (e.g., LSTM / Prophet) that predicts passenger demand per route, per time slot.

- The system shall generate optimized schedule and fleet-allocation recommendations from the forecast output.
- The system shall expose REST APIs that serve real-time passenger information to a commuter-facing mobile application.
- The system shall provide an operator dashboard displaying live occupancy, forecasted demand, and fleet status.
- The system shall send notifications/alerts to operators when overcrowding is predicted on a route.
- The system shall be containerized so that every component (frontend, backend, database, ML engine) can be deployed consistently across environments.

#### 1.4 Expected Outcomes

- Improved service reliability across all routes and time slots.
- Reduced passenger waiting time at stops and stations.
- Better utilization of the existing bus/train fleet.
- Increased commuter satisfaction through predictable, less-crowded travel.
- Lower operational costs from optimized fleet deployment.
- Accurate, data-driven demand forecasting replacing static timetables.
- Enhanced urban mobility and reduced road congestion.
- A sustainable, scalable model for public-transportation management.

## 2. Project Structure Design

The repository is organized using a modular, layered structure so that the frontend, backend, machine-learning components, data assets, and deployment configuration remain independently maintainable while still being version-controlled together as a single monorepo. This structure directly supports the layered system architecture described in Section 7 (Docker Environment Design).



Fig. 2 — Project Repository Folder Structure

### 2.1 Justification of Major Folders and Files

| Folder / File       | Purpose                                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| backend/app/api/    | Contains REST API route handlers exposed to the frontend and external consumers (e.g., /forecast, /schedule, /fleet). |
| backend/app/models/ | Database ORM models representing ridership records, routes, stops, vehicles, and forecast outputs.                    |



|                               |                                                                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| backend/app/services/         | Business logic layer that orchestrates data retrieval, calls to the ML engine, and schedule-optimization logic.                                          |
| backend/app/ml/               | Houses the demand-forecasting model code — training scripts, inference wrapper, and saved model artifacts.                                               |
| backend/tests/                | Unit and integration tests for API endpoints and ML inference correctness.                                                                               |
| frontend/src/components/      | Reusable UI components (charts, route cards, alert banners) used across pages.                                                                           |
| frontend/src/pages/           | Top-level views such as the Operator Dashboard and Commuter App screens.                                                                                 |
| frontend/src/services/        | API-client wrappers used by the frontend to call backend REST endpoints.                                                                                 |
| data/raw/ and data/processed/ | Separates untouched source data (GPS logs, ticketing exports, weather archives) from cleaned datasets used for model training, ensuring reproducibility. |
| notebooks/                    | Jupyter notebooks used for exploratory data analysis and model experimentation before code is finalized into backend/app/ml/.                            |
| docker/                       | Holds Dockerfile.backend, Dockerfile.frontend and docker-compose.yml so all container definitions are centralized and easy to locate.                    |
| docs/                         | Architecture diagrams, workflow diagrams, and the technical report itself.                                                                               |
| .github/workflows/            | CI/CD pipeline definitions that run automated tests and builds on every push/PR.                                                                         |
| .gitignore                    | Excludes build artifacts, virtual environments, node_modules, .env secrets, and large data/model files from version control.                             |
| README.md                     | Entry-point documentation describing setup, run instructions, and project overview for any new contributor.                                              |

## 3. Git Repository Initialization

### 3.1 Repository Initialization Steps

The repository was initialized locally and then linked to a remote hosting service (GitHub) so that the codebase could be version-controlled and collaboratively developed. The steps followed are:

Create the project root folder `smart-transit-forecast/` and navigate into it.

Run `git init` to initialize an empty local Git repository — this creates the hidden `.git/` directory that stores all version history, references, and configuration.

Create a `.gitignore` file listing files/folders that must never be tracked (e.g., `node_modules/`, `__pycache__`, `.env`, `*.pyc`, `/data/raw/*.csv`, model checkpoints).

Create an initial `README.md` describing the project purpose, setup steps, and folder structure.

Stage all initial files using `git add .` and record the first commit using `git commit -m "Initial commit: project scaffolding"`.

Create a new empty repository on GitHub and link it to the local repository using `git remote add origin <repository-url>`.

Push the initial commit to the remote using `git push -u origin main`, establishing the main branch as the upstream tracking branch.

```
$ mkdir smart-transit-forecast && cd smart-transit-forecast
$ git init
$ git add .gitignore README.md
$ git commit -m "Initial commit: project scaffolding"
$ git remote add origin https://github.com/<user>/smart-transit-forecast.git
$ git branch -M main
$ git push -u origin main
```

### 3.2 Purpose of Essential Git Files

| File / Directory        | Purpose                                                                                                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>.git/</code>      | Hidden directory created by <code>git init</code> ; stores the entire object database, commit history, branch references, and repository configuration. |
| <code>.gitignore</code> | Specifies intentionally untracked files (build output, secrets, large data files) so they are never staged or committed.                                |

|                                 |                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| README.md                       | Provides an overview of the project, setup instructions, and usage guidance for any developer cloning the repository.    |
| docker-compose.yml              | Tracked in version control so that the exact multi-container runtime configuration is reproducible by every team member. |
| requirements.txt / package.json | Pin backend (Python) and frontend (Node.js) dependencies so environments are reproducible across machines.               |

## 4. Git Branching Strategy

### 4.1 Selected Branching Model — Gitflow

A Gitflow-inspired branching strategy was selected for this project, using five categories of branches: main, develop, feature/\*, release/\*, and hotfix/\*. Each branch has a clearly defined role in the software lifecycle, as shown below.

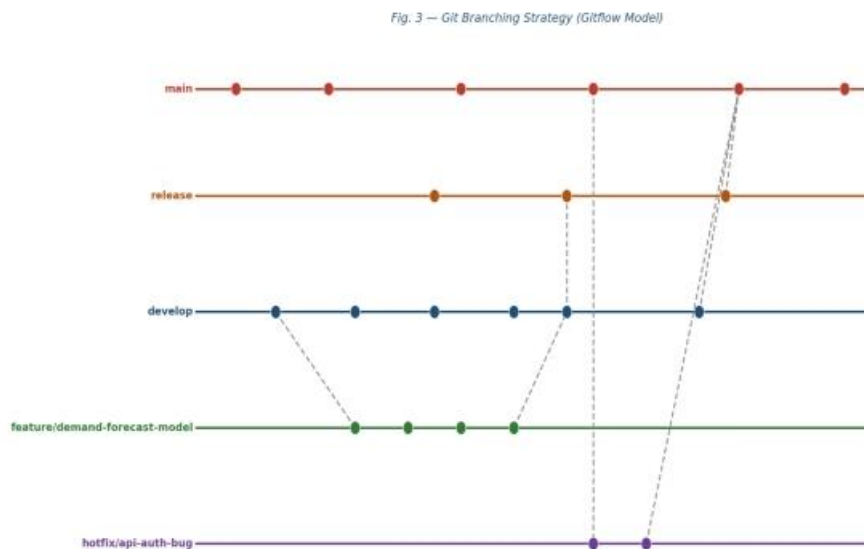


Fig. 3 — Git Branching Strategy (Gitflow Model)

| Branch    | Purpose                                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| main      | Always reflects the production-ready, deployed state of the application. Only release and hotfix branches are merged here.                                                |
| develop   | Integration branch where all completed features are merged and tested together before a release is cut.                                                                   |
| feature/* | Short-lived branches created from develop for a single feature (e.g., feature/demand-forecast-model, feature/operator-dashboard). Merged back into develop when complete. |

|           |                                                                                                                                                                 |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| release/* | Created from develop when preparing a new production release; used for final QA, bug-fixing, and version tagging before merging into both main and develop.     |
| hotfix/*  | Created directly from main to urgently patch a production issue (e.g., hotfix/api-auth-bug), then merged into both main and develop to keep history consistent. |

## 4.2 Justification for the Assigned Project

The Smart Public Transportation Demand Forecasting System involves several independently developed modules — the ML forecasting engine, the scheduling optimizer, the operator dashboard, and the commuter API — that are built in parallel by different contributors or at different times. Gitflow is well suited to this project because:

- Feature isolation: Each module (e.g., the LSTM forecasting model, the fleet-optimization logic) can be developed on its own feature/\* branch without destabilizing the shared develop branch.
- Safe integration: The develop branch acts as a continuously integrated staging area where the ML engine, backend API, and frontend dashboard are verified to work together before release.
- Controlled releases: Because this is a transit-critical system, release/\* branches allow a final testing and stabilization phase before changes reach commuters and operators in main.
- Rapid incident response: If a critical bug is found in production (e.g., an authentication failure in the live API), a hotfix/\* branch allows an immediate, isolated fix without waiting for the next full release cycle.

## 5. Version Control Workflow

The standard Git workflow followed for every unit of work in the project — from picking up a task to deploying it — is illustrated below, followed by an explanation of each operation and how conflicts were resolved when they occurred.

Fig. 4 — Git Version Control Workflow

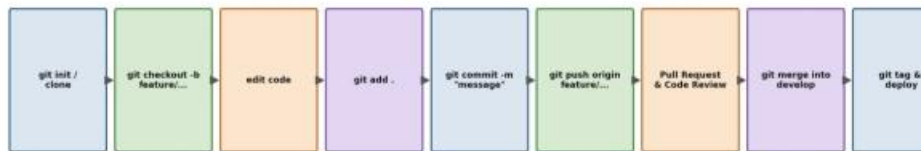


Fig. 4 — Git Version Control Workflow

## 5.1 Explanation of Each Git Operation

| Operation                                     | Purpose                                                                                                                                      |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| git checkout -b feature/demand-forecast-model | Creates and switches to a new isolated branch off develop so the demand-forecasting model can be built without affecting other work.         |
| git add <files>                               | Stages specific changed files (e.g., ml/train_model.py, ml/inference.py) so only intended changes are included in the next commit.           |
| git commit -m "message"                       | Records a permanent snapshot of the staged changes with a descriptive message explaining what changed and why.                               |
| git push origin feature/demand-forecast-model | Uploads the local branch and its commits to the remote GitHub repository so teammates can review it.                                         |
| Pull Request + Code Review                    | A PR is opened from the feature branch into develop; at least one teammate reviews the diff, checks CI results, and approves before merging. |
| git merge (via PR)                            | Integrates the reviewed feature branch into develop, combining both branches' histories.                                                     |
| git pull / git fetch                          | Synchronizes the local repository with the latest changes from the remote before starting new work, reducing the chance of conflicts.        |

|                                           |                                                                                                           |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>git tag v1.0.0 &amp; release</code> | Marks a specific commit on main as an official, deployable version once a release branch has been merged. |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|

## 5.2 Conflict Resolution Example

A merge conflict occurred when two contributors simultaneously modified the same section of `backend/app/services/schedule_service.py` — one adding fleet-reallocation logic and the other adding demand-threshold checks. Git flagged the file as conflicted during the merge of `feature/fleet-optimizer` into `develop`. The conflict was resolved by:

Running `git status` to identify the conflicted file.

Opening the file and manually reviewing the conflict markers (`<<<<<<<`, `=====`, `>>>>>>>`) to understand both sets of changes.

Combining both pieces of logic — the fleet-reallocation function and the demand-threshold check — into a single coherent function, since both were needed.

Removing the conflict markers, staging the resolved file with `git add`, and completing the merge with `git commit`.

Re-running the test suite to confirm both features still worked correctly together before pushing.

## 6. Commit History Analysis

## 6.1 Representative Commit History

The table below presents a representative excerpt of the project's commit history (as would be produced by `git log --oneline --graph`), demonstrating incremental, meaningful development across modules.

| Commit  | Message                                                 | Description                                                      |
|---------|---------------------------------------------------------|------------------------------------------------------------------|
| a1c4e2f | Initial commit: project scaffolding                     | Created base folder structure, README, and .gitignore.           |
| b7f21ab | feat(data): add ridership data ingestion script         | Implemented CSV/GPS log loader for raw ridership data.           |
| C93de10 | feat(ml): implement LSTM demand forecasting model       | Added model architecture, training loop, and evaluation metrics. |
| D4e88c2 | test(ml): add unit tests for forecast accuracy          | Added RMSE/MAE test coverage for the trained model.              |
| E02fa77 | feat(backend): add /forecast REST endpoint              | Exposed model inference through a FastAPI route.                 |
| F56b9de | fix(backend): correct timezone handling in schedule API | Fixed a bug causing incorrect peak-hour predictions.             |
| 10ac3f4 | feat(frontend): build operator dashboard charts         | Added Recharts-based demand visualization components.            |
| 22bd781 | docker: add Dockerfile and docker-compose configuration | Containerized backend, frontend, and database services.          |
| 3ef90aa | merge: integrate feature/fleet-optimizer into develop   | Merged fleet-reallocation logic after conflict resolution.       |
| 47cb650 | release: tag v1.0.0 for production deployment           | Tagged stable release after QA on the release branch.            |

## 6.2 Good Version-Control Practices Reflected in the History

- Atomic commits: Each commit represents one logical change (e.g., adding a single endpoint, fixing one bug), making the history easy to read and, if necessary, to revert.
- Conventional commit prefixes: Prefixes such as feat, fix, test, and docker communicate the type of change at a glance and enable automated changelog generation.



- Descriptive messages: Every message explains what changed and, where relevant, why — avoiding vague messages like “update” or “fix stuff”.
- Traceability: Commit history shows a clear progression from scaffolding to data ingestion to model development to API exposure to containerization, mirroring the actual development lifecycle.
- Contribution to project maintenance: A clean, descriptive history allows future contributors to use git blame and git log to quickly understand why a specific line of code exists, which greatly simplifies debugging and onboarding.

## 7. Docker Environment Design

### 7.1 Why Docker Is Suitable for This Project

The Smart Public Transportation Demand Forecasting System is composed of multiple heterogeneous components: a Python/FastAPI backend with an ML forecasting engine, a React frontend, a PostgreSQL relational database, a Redis cache, and an InfluxDB time-series store for GPS telemetry. Each of these has different runtime dependencies (Python packages, Node.js modules, database binaries) that are difficult to install and keep consistent manually across development, testing, and production machines.

Docker addresses this directly by packaging each component together with its exact dependencies into an isolated, portable container image. This guarantees that the ML model runs against the same Python and library versions everywhere, eliminates "it works on my machine" issues, and allows the entire multi-service system to be started with a single command using Docker Compose.

### 7.2 Application Components Requiring Containerization

| Component                      | Reason for Containerization                                                                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend API + ML Engine        | Requires a specific Python version, ML libraries (TensorFlow/PyTorch, scikit-learn), and consistent inference behaviour across environments.      |
| Frontend (React dashboard/app) | Needs Node.js build tooling for compilation and an Nginx runtime for serving static assets in production.                                         |
| PostgreSQL Database            | Stores relational data (routes, stops, users); containerizing it allows a disposable, reproducible database instance for development and testing. |
| Redis Cache                    | Caches frequently requested forecast results to reduce latency; runs as an isolated in-memory service.                                            |
| InfluxDB (Time-Series DB)      | Stores high-frequency GPS/AVL telemetry; benefits from an isolated, independently scalable container.                                             |
| Forecast Worker                | A background/scheduled service that periodically retrains or re-runs the forecasting model; isolated so it does not block the main API container. |

## 8. Dockerfile Development

### 8.1 Backend Dockerfile

The backend Dockerfile uses a slim Python base image, installs dependencies from requirements.txt, copies the application code (including the ML inference module), and exposes the FastAPI service on port 8000.

```
# docker/Dockerfile.backend
FROM python:3.11-slim

WORKDIR /app

COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ .

ENV PYTHONUNBUFFERED=1
EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

### 8.2 Frontend Dockerfile (Multi-Stage Build)

The frontend uses a multi-stage build: the first stage compiles the React application using Node.js, and the second stage serves only the resulting static files through a lightweight Nginx image — keeping the final image small and free of build tooling.

```
# docker/Dockerfile.frontend
FROM node:20-alpine AS build
WORKDIR /app
COPY frontend/package*.json ./
RUN npm install
COPY frontend/ .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

### 8.3 Purpose of Each Dockerfile Instruction

| Instruction | Purpose                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------|
| FROM        | Specifies the base image (e.g., python:3.11-slim, node:20-alpine) that provides the runtime environment for the container. |
| WORKDIR     | Sets the working directory inside the container for all subsequent instructions and the running process.                   |

|                                  |                                                                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| COPY                             | Copies files from the build context (host machine) into the image — first dependency manifests, then source code, to leverage Docker's layer caching. |
| RUN                              | Executes a command during image build, such as installing dependencies with <code>pip install</code> or <code>npm install</code> .                    |
| ENV                              | Sets environment variables inside the container, e.g., disabling Python output buffering for real-time logs.                                          |
| EXPOSE                           | Documents which port the containerized service listens on (informational; actual publishing is done via <code>-p</code> or docker-compose ports).     |
| CMD                              | Defines the default command executed when the container starts (e.g., launching the Uvicorn or Nginx server).                                         |
| Multi-stage<br>FROM ... AS build | Separates the heavy build environment (Node.js, npm) from the lean production runtime (Nginx), significantly reducing final image size.               |

## 9. Docker Compose Design

Docker Compose is used to orchestrate all containers of the system as a single multi-service application, defining how they are built, networked, and provided with persistent storage.

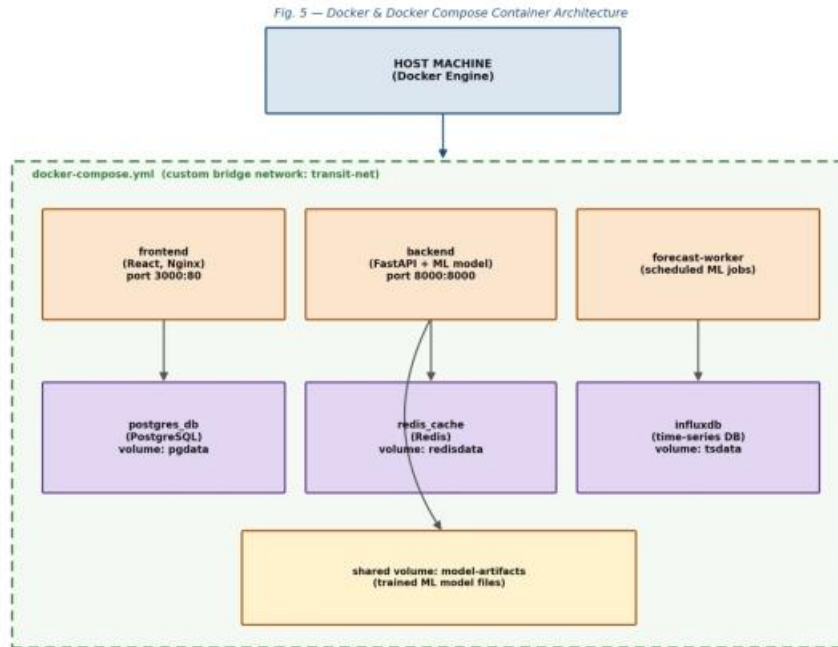


Fig. 5 — Docker & Docker Compose Container Architecture

```
# docker/docker-compose.yml
version: "3.9"
services:
  frontend:
    build:
      context: ..
      dockerfile: docker/Dockerfile.frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks: [transit-net]

  backend:
    build:
      context: ..
      dockerfile: docker/Dockerfile.backend
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://transit:transit@postgres_db:5432/transitdb
      - REDIS_URL=redis://redis_cache:6379/0
    volumes:
      - model-artifacts:/app/ml/artifacts
    depends_on:
      - postgres_db
      - redis_cache
    networks: [transit-net]

  postgres_db:
    image: postgres:16
    environment:
      - POSTGRES_USER=transit
      - POSTGRES_PASSWORD=transit
      - POSTGRES_DB=transitdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks: [transit-net]

  redis_cache:
    image: redis:7-alpine
    volumes:
      - redisdata:/data
    networks: [transit-net]

  influxdb:
    image: influxdb:2.7
    volumes:
      - tsdata:/var/lib/influxdb2
    networks: [transit-net]

volumes:
  pgdata:
  redisdata:
  tsdata:
  model-artifacts:

networks:
  transit-net:
    driver: bridge
```

## 9.1 Services, Networking, Volumes, and Dependencies

| Element                            | Explanation                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Services                           | frontend, backend, postgres_db, redis_cache, and influxdb are each defined as an independent service, built or pulled as a separate image and run as a separate container.                                        |
| networks: transit-net              | A custom bridge network so all containers can resolve each other by service name (e.g., the backend connects to postgres_db, not an IP address), while remaining isolated from other Docker networks on the host. |
| volumes: pgdata, redisdata, tsdata | Named volumes that persist database and cache data outside the container lifecycle, so data survives container restarts or rebuilds.                                                                              |

|                          |                                                                                                                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| volumes: model-artifacts | A shared volume that stores trained ML model files so the backend service can load the latest model without rebuilding the image each time it is retrained.               |
| environment              | Environment variables (e.g., DATABASE_URL, REDIS_URL) inject configuration into containers at runtime, keeping credentials and connection strings out of the source code. |
| depends_on               | Declares startup ordering — e.g., backend waits for postgres_db and redis_cache to be created before starting — ensuring dependent services are available.                |
| ports                    | Maps container-internal ports to host ports (e.g., 3000:80 for the frontend, 8000:8000 for the backend API) so services are reachable from outside Docker.                |

## 10. Container Deployment and Testing

### 10.1 Deployment Steps

The complete system is built and started with a single Docker Compose command, which builds each service's image (if not already built) and starts all containers in the correct dependency order on the shared transit-net network.

```
$ cd smart-transit-forecast/docker
$ docker compose build
$ docker compose up -d
$ docker compose ps
```

| NAME        | IMAGE           | STATUS       | PORTS                  |
|-------------|-----------------|--------------|------------------------|
| frontend    | docker-frontend | Up 2 minutes | 0.0.0.0:3000->80/tcp   |
| backend     | docker-backend  | Up 2 minutes | 0.0.0.0:8000->8000/tcp |
| postgres_db | postgres:16     | Up 2 minutes | 5432/tcp               |
| redis_cache | redis:7-alpine  | Up 2 minutes | 6379/tcp               |
| influxdb    | influxdb:2.7    | Up 2 minutes | 8086/tcp               |

### 10.2 Testing Procedure and Evidence of Successful Execution

Container health check: `docker compose ps` confirms every service is in the "Up" state without restart loops.

API verification: `curl http://localhost:8000/forecast?route=12A` returns a 200 OK response with a predicted demand value, confirming the backend and ML engine are functioning inside the container.

Database connectivity test: The backend logs show a successful connection to `postgres_db` on startup, and a sample query (e.g., listing routes) returns expected rows.

Frontend accessibility: Navigating to `http://localhost:3000` in a browser loads the operator dashboard, which in turn successfully calls the backend API and renders the demand-forecast chart.

Log inspection: `docker compose logs -f backend` is used to confirm there are no unhandled exceptions during startup or during a sample forecast request.

Persistence test: The stack is stopped and restarted with `docker compose down && docker compose up -d`; because `pgdata` and `tsdata` are named volumes, previously stored routes and telemetry data are confirmed still present, verifying that volumes correctly persist state.

Screenshots of the `docker compose ps` output, the browser dashboard, and sample API responses are included in the appendix/screenshots section of the submitted report to provide direct visual evidence of successful deployment.



## 11. Benefits of Git and Docker

| Benefit            | How It Applies to This Project                                                                                                                                                                         |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Collaboration      | Multiple contributors work in parallel on the ML model, backend API, and frontend dashboard using isolated feature branches, then integrate safely through reviewed pull requests into develop.        |
| Version Management | Every change to the forecasting model, API, or UI is tracked with a full, auditable history, enabling rollback to any previous working state if a regression is introduced.                            |
| Portability        | Docker images package the ML engine, backend, and frontend with all dependencies, so the exact same system runs identically on a developer laptop, a CI runner, or a cloud production server.          |
| Deployment         | docker compose up (or an equivalent orchestrated deployment) brings up the entire multi-service transit platform in minutes, drastically simplifying release to production.                            |
| Scalability        | Individual containers (e.g., the backend or forecast-worker) can be scaled horizontally behind a load balancer during peak commuting hours without needing to scale the entire monolith.               |
| Maintainability    | Clear branch structure and descriptive commit history make it straightforward to locate when and why a specific piece of forecasting or scheduling logic was introduced, easing long-term maintenance. |

## 12. Challenges and Improvements

### 12.1 Challenges Encountered

- Merge conflicts arose when multiple contributors modified the same scheduling-logic file simultaneously, requiring careful manual reconciliation (see Section 5.2).
- Coordinating environment variables and secrets (e.g., database credentials, API keys for the weather service) across containers without committing them to version control required setting up a separate, git-ignored .env file.
- Ensuring the ML model's Python dependencies (e.g., specific TensorFlow/PyTorch versions) matched exactly between the training notebooks and the containerized inference service initially caused version-mismatch errors.
- Keeping Docker image sizes small while still including all ML libraries required adopting a multi-stage build and slim base images.
- Persisting trained model artifacts across container rebuilds required introducing a dedicated named volume rather than baking the model into the image.

### 12.2 Suggested Improvements and Future Enhancements

Adopt a full CI/CD pipeline (via `.github/workflows/`) that automatically runs tests, builds Docker images, and pushes them to a container registry on every merge to main.

Introduce automated model-retraining pipelines triggered on a schedule, with model-performance monitoring to detect forecast drift over time.

Add container orchestration (e.g., Kubernetes) for production to support auto-scaling of the backend and forecast-worker services during peak demand periods.

Extend the Git workflow with mandatory branch-protection rules and required status checks before merging into develop and main.

Incorporate real-time streaming (e.g., Kafka) more deeply into the pipeline to reduce forecast latency from near-real-time to true real-time.

Add centralized logging and monitoring (e.g., Prometheus/Grafana) across all containers for proactive operational visibility.

### Expected Deliverables

- Technical report (10 pages) — this document.
- Git repository with complete commit history.
- Source code of the assigned project (Smart Public Transportation Demand Forecasting System).
- Dockerfile(s) for backend and frontend.

- Docker Compose file (docker-compose.yml).
- Screenshots of Git operations and Docker execution.
- Architecture and workflow diagrams (Figures 1–5 of this report).